

Ex. No. : 3
16.12.2025

IMPLEMENTATION OF DEADLOCKS Date:

AIM

To implement Banker's Algorithm for deadlock avoidance.

PROCEDURE

1. Define allocation, maximum, and available resource matrices.
2. Compute the need matrix.
3. Check whether the system is in a safe state.

PROGRAM

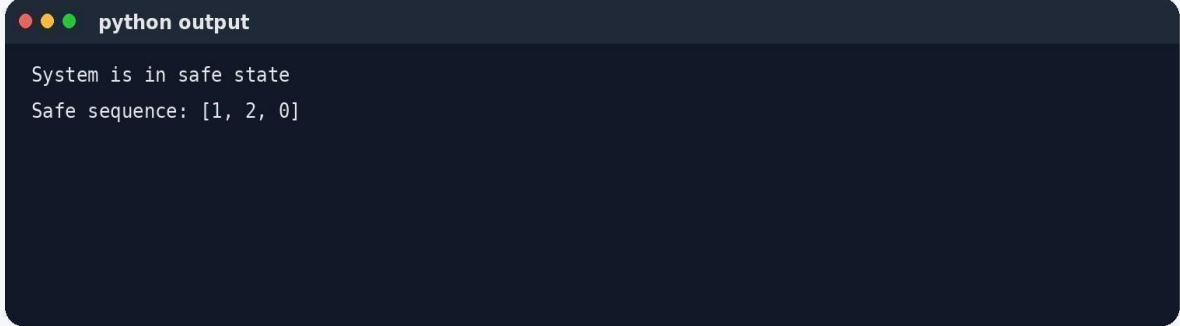
```
alloc = [[0, 1, 0], [2, 0, 0], [3, 0, 2]] maxm = [[7, 5, 3], [3, 2, 2], [9, 0, 2]] avail = [3, 3, 2]
```

```
n = len(alloc) m = len(avail)
need = [[maxm[i][j] -
alloc[i][j] for j in range(m)]:
    for i in range(n)]:
    finish = [False] * n
    safe_seq = []
while len(safe_seq) < n: found =
    False for i in range(n):
    if not finish[i] and all(need[i][j] <= avail[j] for
j in range(m)):
        for j in range(m):
            avail[j] +=
            alloc[i][j]
        safe_seq.append(i)
        finish[i] = True
        found = True
if not found:
    break

if len(safe_seq) == n:
    print("System is in safe
state")
print("Safe sequence:", safe_seq)
else: print("System is not in
safe state")
```

OUTPUT

Practice 3 - Sample Output



```
python output  
System is in safe state  
Safe sequence: [1, 2, 0]
```

RESULT

Thus, the given programs were implemented successfully.

The expected outputs were obtained and the required concepts were verified.